

# tcpdump Cheat Sheet

Watch the packets going in and out of a network interface, live or saved to a file for Wireshark.

## Using it

```
sudo tcpdump [options] [filter] ← Ctrl-C stops it and prints how many packets it saw
sudo tcpdump -nn -i main port 53 ← DNS traffic on the bridge, no name lookups
```

| Option           | Means                                                             | Option           | Means                                    |
|------------------|-------------------------------------------------------------------|------------------|------------------------------------------|
| <b>-i eno1</b>   | Interface to listen on ( <b>main</b> , <b>eno1</b> , <b>any</b> ) | <b>-A / -X</b>   | Show payload as text / as hex + text     |
| <b>-n</b>        | Don't turn IPs into names (faster)                                | <b>-v, -vv</b>   | More detail: TTL, checksums, DHCP fields |
| <b>-nn</b>       | Also keep port numbers ( <b>22</b> not <b>ssh</b> )               | <b>-s 0</b>      | Whole packets (already the default now)  |
| <b>-c 100</b>    | Stop after 100 packets                                            | <b>-e</b>        | Show MAC addresses and VLAN tags         |
| <b>-w f.pcap</b> | Write packets to a file (shows nothing)                           | <b>-r f.pcap</b> | Read a saved file (no sudo needed)       |

## Filters (what to capture)

| Filter                    | Matches                       | Filter                  | Matches                     |
|---------------------------|-------------------------------|-------------------------|-----------------------------|
| <b>host 192.168.1.10</b>  | To or from that IP            | <b>tcp / udp / icmp</b> | Only that protocol          |
| <b>src 192.168.1.10</b>   | From it ( <b>dst</b> : to it) | <b>arp</b>              | ARP (who-has / is-at)       |
| <b>port 443</b>           | Either side uses port 443     | <b>vlan 20</b>          | VLAN 20 tagged frames       |
| <b>dst port 22</b>        | Going to port 22              | <b>ether host MAC</b>   | To or from that MAC address |
| <b>net 192.168.1.0/24</b> | Any address in that subnet    | <b>and / or / not</b>   | Combine (&&    ! also work) |

Quote filters with parentheses: **'host 192.168.1.10 and (port 80 or port 443)'**

## Reading the output (a TCP connection to SSH)

```
$ sudo tcpdump -nn -i main host 192.168.1.50 and port 22
10:02:11.381204 IP 192.168.1.50.51734 > 192.168.1.10.22: Flags [S], seq 3012745521, win 64240, options [...], ...
10:02:11.381390 IP 192.168.1.10.22 > 192.168.1.50.51734: Flags [S.], seq 1938270044, ack 3012745522, win 65160, ...
10:02:11.381512 IP 192.168.1.50.51734 > 192.168.1.10.22: Flags [.], ack 1, win 502, length 0
10:02:11.384020 IP 192.168.1.10.22 > 192.168.1.50.51734: Flags [P.], seq 1:42, ack 1, win 510, length 41
```

| Part                      | Means                                        | Part                | Means                                       |
|---------------------------|----------------------------------------------|---------------------|---------------------------------------------|
| <b>10:02:11.381204</b>    | Time seen (microseconds)                     | <b>seq / ack</b>    | Byte counters; from 1 after the handshake   |
| <b>IP / IP6</b>           | Protocol ( <b>-e</b> puts MACs, VLAN first)  | <b>seq 1:42</b>     | This packet carries bytes 1-41              |
| <b>a.b.c.d.51734 &gt;</b> | Sender IP, then <b>.port</b> ; then receiver | <b>win / length</b> | Receiver's free buffer / bytes of data      |
| Flags                     | Means                                        | Flags               | Means                                       |
| <b>[S]</b>                | SYN: "can we talk?" (new connection)         | <b>[F.]</b>         | FIN: "I'm done" (polite close)              |
| <b>[S.]</b>               | SYN-ACK: "yes, let's talk"                   | <b>[R] / [R.]</b>   | RST: rejected / slammed shut (port closed?) |
| <b>[.]</b>                | ACK only: "got it"                           | <b>[P.]</b>         | PUSH: carrying data, deliver it now         |

```
10:05:00.112233 IP 192.168.1.10 > 192.168.1.1: ICMP echo request, id 3, seq 1, length 64
10:05:00.112601 IP 192.168.1.1 > 192.168.1.10: ICMP echo reply, id 3, seq 1, length 64 ← a ping
```

## Handy recipes

| Command                                                              | Does                                        |
|----------------------------------------------------------------------|---------------------------------------------|
| <b>sudo tcpdump -nv -i main port 67 or port 68</b>                   | Watch DHCP requests and offers              |
| <b>sudo tcpdump -nn -i main port 53</b>                              | Every DNS question and answer               |
| <b>sudo tcpdump -nni main 'port 8080 and tcp[tcpflags]==tcp-syn'</b> | Who is opening connections to port 8080     |
| <b>sudo tcpdump -nn -c 3 -i main tcp port 80</b>                     | First 3 packets = a handshake: [S] [S.] [.] |
| <b>sudo tcpdump -i eno1 -c 50000 -w /tmp/cap.pcap</b>                | Save to a file to open in Wireshark         |
| <b>sudo tcpdump -nn -i main not port 22</b>                          | Everything except your own SSH session      |

**Remember:** tcpdump needs **sudo**. On a bridge, capture on **main** to see the host's own traffic, or on **eno1** to see everything crossing the physical wire (host and VMs). Add **-nn** so DNS lookups don't slow it down or add packets of their own.